La boucle For en JavaScript

La boucle `for` est une structure de contrôle qui permet d'exécuter un bloc de code un nombre déterminé de fois. Elle est particulièrement utile pour parcourir des tableaux ou répéter des actions dans les jeux vidéo. Cela permet de réduire la répétition de code et de rendre le programme plus efficace.

Syntaxe de base

La boucle `for` a la syntaxe suivante

```
for (initialisation; condition; incrémentation) {
    // Code à exécuter à chaque itération
}
```

Les trois parties de la boucle for :

1. **Initialisation** : Déclaration et initialisation de la variable de contrôle (souvent `i`)
2. **Condition** : La boucle continue tant que cette condition est vraie
3. **Incrémentation** : Modification de la variable de contrôle à chaque itération

Arrêter la boucle ou sauter des itérations avec `break` et `continue`

L'instruction `break` permet de sortir complètement de la boucle, tandis que `continue` saute à l'itération suivante.

```
// Chercher un ennemi spécifique
for (let i = 0; i < aEnnemis.length; i++) {
    if (aEnnemis[i].estMort) {
        continue; // Passer au suivant, le rester du code n'est pas exécuté
    }

    if (aEnnemis[i].type === "boss") {
        console.log("Boss trouvé à l'index " + i);
        break; // Arrêter la recherche
    }
}
```

Exemple simple : Compter les points de vie

```
let nPointsDeVie = 100;
let nbDeCoupsRecus = 5;
// Diminuer les points de vie de 10 à chaque coups, 5 fois
// On continue tant que i est inférieur à nbDeCoupsRecus. Si cela n'est plus le
cas, on sort de la boucle.
for (let i = 0; i < nbDeCoupsRecus; i++) {
    nPointsDeVie -= 10;
```

```
    //Afficher une animation de dégât ici à chaque coup
    console.log("Tour " + (i + 1) + ": Points de vie = " + nPointsDeVie);
}
// Résultat: 50 points de vie restants
```

Parcourir un tableau avec for

La boucle `for` est idéale pour parcourir tous les éléments d'un tableau.

```
let aEnnemis = ["gobelin", "orc", "dragon", "sorcier"];

for (let i = 0; i < aEnnemis.length; i++) {
    console.log("Ennemi " + i + ": " + aEnnemis[i]);
}
```

Exemples pratiques

1. Dessiner une grille de jeu

On peut utiliser deux boucles `for` imbriquées pour dessiner une grille de jeu sur un canvas HTML. La première boucle parcourt les lignes, et la seconde boucle parcourt les colonnes.

```
let canvas = document.getElementById("monCanvas");
let contexte = canvas.getContext("2d");

// Dessiner une grille 8x8 pour un jeu de plateau
let nTailleCase = 50;

for (let ligne = 0; ligne < 8; ligne++) {
    for (let colonne = 0; colonne < 8; colonne++) {
        let x = colonne * nTailleCase;
        let y = ligne * nTailleCase;

        // Alterner les couleurs comme un échiquier
        if ((ligne + colonne) % 2 === 0) {
            contexte.fillStyle = "#F0D9B5"; // Beige
        } else {
            contexte.fillStyle = "#B58863"; // Marron
        }

        contexte.fillRect(x, y, nTailleCase, nTailleCase);
    }
}
```

Exemple plus avancé avec un tableau de données

```
```

```
let niveau = [
    ["herbe", "herbe", "eau", "herbe"],
    ["herbe", "pierre", "pierre", "herbe"],
    ["herbe", "herbe", "herbe", "arbre"],
    ["eau", "eau", "herbe", "herbe"]
];

for (let y = 0; y < niveau.length; y++) {
    for (let x = 0; x < niveau[y].length; x++) {

        let typeCase = niveau[y][x];

        console.log("Case (" + x + ", " + y + "): " + typeCase);
        if (typeCase === "herbe") {
            contexte.fillStyle = "green";
        } else if (typeCase === "eau") {
            contexte.fillStyle = "blue";
        } else if (typeCase === "pierre") {
            contexte.fillStyle = "gray";
        } else if (typeCase === "arbre") {
            contexte.fillStyle = "darkgreen";
        }
        contexte.fillRect(x * nTailleCase, y * nTailleCase, nTailleCase,
nTailleCase);
    }
}
```

2. Déplacer plusieurs projectiles

```
let aProjectiles = [
    {x: 100, y: 300, vitesse: 5},
    {x: 150, y: 250, vitesse: 7},
    {x: 200, y: 400, vitesse: 6}
];

function deplacerProjectiles() {
    // Parcourir tous les projectiles et les déplacer
    for (let i = 0; i < aProjectiles.length; i++) {
        aProjectiles[i].x += aProjectiles[i].vitesse;

        // Dessiner le projectile
        contexte.fillStyle = "yellow";
        contexte.fillRect(aProjectiles[i].x, aProjectiles[i].y, 10, 5);

        // Supprimer le projectile s'il sort de l'écran
        if (aProjectiles[i].x > canvas.width) {
            aProjectiles.splice(i, 1);
            i--; // Ajuster l'index après suppression
        }
    }
}
```

Si on doit retirer un élément, il est préférable de parcourir le tableau à l'envers pour éviter de sauter des éléments.

```
for (let i = aProjectiles.length - 1; i >= 0; i--) {
    if (aProjectiles[i].x > canvas.width) {
        aProjectiles.splice(i, 1);
    }
}
```

3. Générer un niveau procédural

```
let aTypesBlocs = ["herbe", "pierre", "eau", "arbre"];
let niveau = [];
let nLargeurNiveau = 20;
let nHauteurNiveau = 15;

// Générer un niveau aléatoire
for (let y = 0; y < nHauteurNiveau; y++) {
    for (let x = 0; x < nLargeurNiveau; x++) {
        let nIndexAleatoire = Math.floor(Math.random() * aTypesBlocs.length);
        let sTypeBloc = aTypesBlocs[nIndexAleatoire];

        // Définir la couleur selon le type de bloc
        switch(sTypeBloc) {
            case "herbe":
                contexte.fillStyle = "green";
                break;
            case "pierre":
                contexte.fillStyle = "gray";
                break;
            case "eau":
                contexte.fillStyle = "blue";
                break;
            case "arbre":
                contexte.fillStyle = "darkgreen";
                break;
        }

        // Dessiner le bloc
        contexte.fillRect(x * 20, y * 20, 20, 20);
        niveau.push(sTypeBloc);//Peut servir pour vérifier les collisions plus
tard
    }
}
```

4. Vérifier les collisions entre le personnage et plusieurs obstacles

```
function verifierCollisions(personnage) {
    for (let i = 0; i < niveau.length; i++) {
        let typeBloc = niveau[i];
        // Vérifier si le personnage entre en collision avec le bloc
        if (estEnCollision(personnage, typeBloc)) {
            console.log("Collision avec le bloc de type : " + typeBloc);
        }
    }
}

function estEnCollision(personnage, bloc) {
    if (bloc === "eau" || bloc === "arbre") {
        return true; // Exemple simplifié
    }

    return false; // Exemple simplifié
}
```